ШАГ 0

Проверка адреса и доступности сайта

GEO-аудит · Подробный алгоритм · Редакция 1.0 · 31.10.2026

Этот документ описывает подготовку сайта к ограниченному чтению на шаге 1. В него включены исправления исходного алгоритма и уточнения для Python-реализации. Исходный PDF сохранён. Численные ограничения ниже являются начальными настройками проекта и могут изменяться с версией конфигурации.

0.1. Назначение и результат

Получить однозначный адрес входной страницы, проверить реальную доступность HTML, определить допустимые хосты и сохранить доказательства всех попыток. Отдельно ответить на вопросы: «получена ли страница?», «исправен ли HTTPS?» и «можно ли передать результат шагу 1?».

На этом этапе нет генерации вопросов, обращений к платным моделям и оценки GEO-видимости. Исполнитель: Python и HTTPX. Пользователь вводит адрес; библиотечные функции возвращают данные и не вызывают input().

0.2. Что исправлено относительно исходника

Исходная формулировка	Рабочая редакция
Всегда привести адрес к https://домен/	Сохранить путь и query; при отсутствии схемы начать с HTTPS
Главная должна вернуть 200	Проверить ответ, тип и характер содержимого; JS-оболочку передать на диагностику
HTTPS не открылся - сайт недоступен	При подходящей транспортной ошибке отдельно проверить HTTP и сохранить проблему HTTPS
Любой другой домен - ошибка	Различать наблюдаемый www-алиас и переход на неподтверждённый хост
Выход - адрес и домен	Выход - site.json, доказательства, статусы, причины и область дальнейшего чтения

0.3. Порядок работы

Ввод и конфигурация → разбор URL → проверка назначения → выбор схемы → ограниченный GET и редиректы → классификация ответа → сохранение site.json → передача результата шагу 1.

Доступность по HTTP допускает чтение публичной страницы. Ошибка HTTPS остаётся отдельной находкой. Успешный шаг 0 не означает, что robots.txt разрешает автоматический обход: это проверяется на шаге 1.

Навигация

Далее: вход и нормализация; политика HTTP/HTTPS; границы редиректов; сетевые ограничения; пригодность страницы; контракты данных; последовательность исполнения; приёмочные сценарии; параметры и источники.

1. Вход и нормализация URL

1.1. Входные данные

Обязательный параметр - строка input_url. Дополнительные: allow_http_fallback, явно разрешённые хосты, таймауты, лимиты, каталог результатов и идентификатор конфигурации. При повторном запуске сохраняется новый run_id; связь с предыдущей попыткой допускается через parent_run_id.

Поле	Назначение
input_url	Исходная строка без потери пользовательского намерения
normalized_url	Первый адрес для запроса после безопасного разбора
input_scheme_explicit	Указал ли пользователь HTTP/HTTPS самостоятельно
effective_url	Конечная успешно проверенная страница, если она получена
origin	Схема, нормализованный хост и эффективный порт
allowed_hosts	Точные хосты, допустимые для страниц этого запуска
entry_path	Исходный путь; по умолчанию /

1.2. Последовательность нормализации

1. Сохранить исходную строку. Удалить только внешние пробелы; отклонить управляющие символы, переносы строк и неоднозначный синтаксис.
2. При отсутствии схемы подготовить HTTPS-кандидат. Адрес вида //example.com/ трактовать как ввод без схемы. Не считать двоеточие в некорректной строке доказательством допустимого URL.
3. Разобрать строку стандартным парсером. Разрешить только http и https, корректный непустой хост и допустимый порт. Встроенные credentials отклонить.
4. Нормализовать хост, IDN и порт; сохранить Unicode-вариант для отображения и ASCII-вариант для сети. Правила IDNA и используемую библиотеку зафиксировать в реализации.
5. Пустой путь заменить на /. Fragment удалить: он не отправляется серверу. Путь и query сохранить; значимые повторяющиеся параметры не переставлять и не удалять.
6. Проверить, что нормализованный адрес стабильно разбирается тем же сетевым стеком. Не выполнять многократное декодирование percent-escape и не менять регистр пути.

1.3. Примеры

Ввод	Первый кандидат
example.com	https://example.com/
http://example.com/ru/?city=msk#prices	http://example.com/ru/?city=msk
https://example.com/Services	Путь /Services сохраняется
ftp://example.com	Ошибка INPUT_SCHEME_UNSUPPORTED
https://user:pass@example.com	Ошибка INPUT_CREDENTIALS_FORBIDDEN

Входной /ru/ задаёт стартовую страницу, но не автоматически запрещает чтение остальных разделов хоста. Ограничение по префиксу пути задаётся отдельно, когда аудит нужен для конкретного раздела. Если глубокий путь вернул 404, автоматически подменять его главной нельзя.

2. Политика HTTP и HTTPS

2.1. Основное правило

При вводе без схемы первым выполняется HTTPS с проверкой сертификата. При явной схеме сначала проверяется указанный адрес. HTTP fallback - отдельная попытка с тем же исходным хостом, путём и query, а не скрытое отключение проверки TLS. По умолчанию fallback включён и отражается в конфигурации. [S1, S2]

Результат первой попытки	Следующее действие
Получен пригодный HTTPS HTML	Принять результат; HTTP специально не запрашивать
Явный HTTP перенаправил на исправный HTTPS	Принять HTTPS; сохранить всю цепочку
TLS certificate failure	Не повторять тот же TLS-запрос; при разрешённом fallback отдельно проверить HTTP
Отказ соединения / ограниченный таймаут HTTPS	После допустимого повтора попробовать HTTP, если остаётся бюджет
Подтверждённый DNS NXDOMAIN	Завершить: смена схемы не исправит отсутствие хоста
Временный DNS-сбой	Только ограниченный сетевой повтор; не дублировать его как «проверку HTTP»
HTTP-статус 401/403/404/429 или 5xx	Обработать статус; автоматически менять схему не нужно
CAPTCHA / подтверждённая страница входа	Сохранить причину; не искать обход другой схемой
HTTPS перенаправил на HTTP	Зафиксировать HTTPS_DOWNGRADE; принять читаемую страницу с предупреждением

2.2. Ограничения fallback

Максимум один альтернативный HTTP-кандидат на входной адрес. Повтор внутри него ограничивается общей политикой временных ошибок. Внешний редирект, небезопасный адрес, неверный ввод и исчерпание лимита никогда не запускают fallback.

Если HTTP перенаправил обратно на ранее неисправный HTTPS, второй круг не начинается: вернуть FALLBACK_NO_USABLE_RESULT. Ошибка первого HTTPS остаётся в attempts. Результаты разных схем не объединяются как эквивалентные страницы без наблюдаемых оснований.

При успешном явном HTTP проверка альтернативного HTTPS, если её ещё не было, выполняется на шаге 1 как отдельная техническая диагностика. Эта диагностика не меняет незаметно выбранную входную страницу и не смешивает правила robots разных origin.

2.3. Как формулировать результат

«Страница прочитана по HTTP. HTTPS-соединение не установлено: ошибка проверки сертификата». Для таймаута: «HTTPS не удалось проверить за отведённое время». Формулировка «HTTPS отсутствует» по одному таймауту недопустима.

Использование HTTP требует предупреждения о незашифрованной передаче: содержимое фиксируется как наблюдение именно этого канала. Ни HTTPX verify=False, ни браузерное ignore_https_errors=True в этой методике не используются.

3. Хосты, редиректы и область анализа

3.1. Первичная область

В allowed_hosts включить нормализованный входной хост и хосты, явно заданные пользователем. Разрешение хоста не снимает запрет внутренних сетей и не добавляет автоматически все его поддомены. Стандартные порты: HTTP 80, HTTPS 443; нестандартный порт требует отдельной конфигурации.

Для каждого origin хранить собственную историю: http://example.com и https://example.com не имеют общих robots-результатов по умолчанию. Смена схемы сохраняется как событие. Публичный CDN не становится страницей сайта только потому, что упомянут в HTML.

3.2. Обработка каждого редиректа

1. Принять ответ с поддерживаемым redirect-статусом 301, 302, 303, 307 или 308. Отсутствующий или некорректный Location зафиксировать как REDIRECT_INVALID_LOCATION.
2. Разрешить относительный Location относительно URL текущего ответа. Нормализовать адрес по тем же правилам, что и вход.
3. Проверить схему, порт, credentials, фактическое публичное назначение, visited-набор и оставшийся hop-бюджет.
4. Если хост уже разрешён - продолжить. Если это допустимый наблюдаемый www-алиас - добавить его с основанием. Во всех остальных случаях остановиться до запроса назначения.
5. Сохранить источник, код, сырой Location, вычисленный адрес, решение и время. Увеличить общий счётчик HTTP-попыток при отправке следующего GET.

3.3. Правило www-алиаса

Автоматический алиас разрешён только для пары host и www.host, когда редирект действительно наблюдался, принадлежность к одной регистрируемой области проверена локальным PSL с private-секцией и ни одна сторона не является самим публичным/частным суффиксом. Простого удаления префикса www недостаточно.

Редирект example.com → shop.example.com не считается разрешённым только из-за общей зоны. tenant-a.example-hosting.com → tenant-b.example-hosting.com тоже не является основанием для объединения компаний. Версию PSL или хэш локального снимка сохранить.

3.4. Внешний переход

Возвратить needs_input, proposed_target_url и причину. next_action содержит явный повторный запуск с allow-host. Библиотека не открывает вопрос в консоли и не выполняет запрос на целевой хост заранее. Если разрешение дано в повторном запуске, цепочка всё равно проверяется полностью.

HTML canonical, hreflang, meta refresh и ссылки не расширяют область шага 0. Потенциальный meta refresh можно записать как наблюдение и задачу последующей диагностики; его не исполнять как неограниченный редирект.

4. Сетевой клиент и лимиты

4.1. Последовательность одного GET

Перед запросом проверить дедлайн, бюджет попыток, назначение и лимит на хост. Применить TLS verification, заданный User-Agent, фиксированный Accept-Language и контролируемую proxy-конфигурацию. Автоматические редиректы и скрытые повторы транспорта отключить. Прокси, cookies, окружение и способ разрешения DNS отражаются в конфигурации, чтобы результаты можно было объяснить.

Получать ответ потоково. Сохранить статус и все значимые заголовки, затем читать тело до установленного предела. Ограничивать как получаемый объём, так и размер после распаковки. Повреждённое или обрезанное тело не считать полным HTML. HEAD не заменяет GET, потому что не даёт содержимое для следующего этапа.

4.2. Начальная конфигурация шага 0

Параметр	Значение и учёт
Общий deadline шага 0	60 секунд; входит в 300 секунд шагов 0-1
Максимум HTTP-попыток шага 0	15; входят в общий лимит 150
Connect / read / write / pool	5 / 15 / 10 / 5 секунд, с общим дедлайном
Redirection hops	До 5; затем REDIRECT_LIMIT
Дополнительные повторы	До 2 на временную ошибку; каждый расходует бюджет
Тело HTML	До 5 MiB после распаковки; до 5 MiB перед ней
Начало запросов на один хост	Не чаще одного в секунду
TLS	Обязательная проверка сертификата

Read timeout HTTPX ограничивает ожидание сетевых данных, поэтому не заменяет внешний дедлайн всей операции. Запрос с медленно поступающими кусками тоже должен завершаться по общему времени. [S3]

4.3. Повторы

Для connect/read-сбоев и 408/429/500/502/503/504 разрешены максимум два дополнительных запроса в остатке бюджета. Задержка: начально 1 и 2 секунды с небольшим jitter. При корректном Retry-After ждать не меньше указанного значения; если ожидание не помещается в дедлайн - закончить с RETRY_DEFERRED.

Неповторяемые случаи: неверный ввод, блок назначения, неверный сертификат, постоянные 4xx, challenge и неверный Location. Повтор HTTP после TLS failure является другой попыткой схемы, а не успешной TLS-проверкой. Для любого исключения сохраняется нормализованный error_code; системный traceback не подменяет понятное сообщение пользователю.

4.4. Назначение соединения

Разрешены только публичные адреса. Проверить IPv4/IPv6, DNS-ответы и каждый новый redirect. localhost, private, link-local, reserved, metadata и IPv4-mapped IPv6 во внутреннюю сеть отклоняются. DNS precheck должен быть связан с фактическим соединением через pinning или контролируемый egress. Повторное разрешение хоста не должно обходить проверку. Проверка одной строки URL не считается защитой от DNS rebinding.

5. Пригодность страницы и итоговый статус

5.1. Классификация полученного ответа

Сначала классифицируется транспорт, затем HTTP-статус, затем содержимое. Страница не становится пригодной только потому, что код равен 200. Классификация содержимого отделена от технической доступности.

Наблюдение	content_state	Действие
200, полноценный HTML	html	Передать шагу 1
200, короткий HTML с признаками приложения	js_candidate	Передать для ограниченной диагностики
200, CAPTCHA/WAF или логин	access_barrier	Сохранить; автоматическое чтение остановить
200, вероятная заглушка/soft-404	suspicious_html	Сохранить уверенность и основания; не считать доказанным описанием бизнеса
204, пустое тело	empty	Пригодной страницы нет
PDF/архив/изображение вместо HTML	non_html	Сообщить, что для этой версии нужен URL страницы
401/403	http_denied	Зафиксировать отказ; обход не запускать
404/410	http_missing	Сохранить ошибку входного пути
429/5xx после допустимых повторов	http_unavailable	Завершить с конкретной причиной

Если Content-Type отсутствует или явно неверен, допустим осторожный sniffing первых 4 KiB. Его результат и основание сохранить. Бинарные данные не декодировать насильно как текст. Для HTML сохранить объявленную и применённую кодировку, число заменённых символов и предупреждение о повреждении декодирования.

5.2. Итоговый статус, переход и причины

status	Условие	can_enter_step1
ok	Получен пригодный HTML без существенных транспортных ограничений	true
partial	Есть HTML, но использован HTTP fallback, есть downgrade или требуется JS-проверка	true
partial	Есть ответ, но это подтверждённый барьер доступа / непригодное содержимое	false
needs_input	Нужен выбор другого адреса или разрешение внешнего хоста	false
failed	Неверный ввод, блок назначения, нет ответа либо устойчивый ошибочный HTTP-статус	false

can_enter_step1 означает допустимость следующего этапа, а не разрешение robots. Неоднозначное поле can_crawl из раннего черновика заменено этим полем. Фактическое разрешение автоматического обхода устанавливает шаг 1.

Приоритет классификации: INPUT/TARGET/OUTPUT-ошибка даёт failed; неподтверждённый внешний переход - needs_input; сетевой провал или окончательный 4xx/5xx - failed; полученный 2xx с непригодным содержимым - partial без перехода. Для пригодного HTML выбирается ok или partial по оставшимся ограничениям.

5.3. Как обрабатывать неоднозначность

Предполагаемый challenge без достаточных признаков остаётся подозрением: его нельзя автоматически назвать блокировкой. Для suspicious_html дальнейшая диагностика допустима только при can_enter_step1=true, отсутствии подтверждённого барьера и сохранённом HTML. next_action должен объяснять конкретное решение. Ошибка сохранения обязательных доказательств превращает завершение в failed с OUTPUT_WRITE_FAILED: положительный результат без проверяемого основания не выдаётся.

5.4. Основные коды причин

Семейство	Примеры и смысл
INPUT_*	Некорректный адрес, схема, порт или credentials
TARGET_*	Непубличное назначение, DNS rebinding, запрещённый переход
DNS_* / NETWORK_*	NXDOMAIN, временный DNS-сбой, отказ соединения, таймаут
HTTPS_*	Ошибка сертификата, TLS handshake, downgrade; не смешивать с HTTP-кодом
REDIRECT_*	Цикл, лимит, некорректный Location, неподтверждённый хост
CONTENT_*	Не HTML, пустое тело, слишком большой ответ, повреждение декодирования
ACCESS_*	Авторизация или подтверждённый challenge
BUDGET_* / OUTPUT_*	Дедлайн, лимит запросов, ошибка записи доказательств

6. Контракт данных и доказательства

6.1. Минимальная структура site.json

```
{
  "schema_version": "1.0",
  "algorithm_version": "step0-1.0",
  "run_id": "fixture-run-001",
  "input_url": "example.com/ru/",
  "normalized_url": "https://example.com/ru/",
  "effective_url": "http://example.com/ru/",
  "input_scheme_explicit": false,
  "allowed_hosts": ["example.com"],
  "status": "partial",
  "can_enter_step1": true,
  "content_state": "html",
  "fallback_used": true,
  "selected_evidence_id": "ev-http-02",
  "reason_codes": ["HTTPS_TLS_CERTIFICATE_ERROR"],
  "attempt_ids": ["req-01", "req-02"],
  "next_action": "continue_step1_with_https_warning"
}
```

Пример учебный: не результат реального аудита. Полная модель также содержит время начала/окончания в UTC, исходную временную зону, версию конфигурации, origin, entry_path, redirect_chain, TLS-наблюдения, счётчики и proposed_target_url при необходимости.

6.2. Запись отдельной попытки

Поля	Обязательное содержание
attempt_id, parent_attempt_id	Устойчивые связи между запросом, повтором и redirect
requested_url, resolved_target	Запрошенный URL и проверенное сетевое назначение
purpose	initial, retry, http_fallback или redirect
status_code, response_headers	Код и повторяющиеся заголовки без потери значений
started_at, elapsed_ms	Время и длительность
transport_error, tls_state	Нормализованный код и отдельный TLS-результат
evidence_id, bytes_read	Связь с телом, размеры и полнота получения

6.3. Правила хранения

В raw/ сохранять тело ответа и манифест с SHA-256, размером, видом представления, URL и временем. Различать content-encoded bytes, распакованный HTML и декодированный текст. Это содержимое HTTP-ответа, а не побайтовая запись TLS/HTTP wire-трафика. Производные данные не перезаписывают исходные.

При отсутствии тела, например TLS failure, evidence ссылается на структурированное наблюдение попытки. Путь к файлу задаётся внутри каталога запуска и не строится напрямую из URL. JSON записывается через временный файл и атомарную замену; site.json ссылается только на уже сохранённые доказательства.

Не сохранять Set-Cookie и другие секреты в общедоступный журнал. Диагностические заголовки хранить по явному списку; редактирование значений отмечать. Сам URL может содержать чувствительный query, поэтому полный адрес доступен в локальных данных, а пользовательский лог использует безопасное представление.

7. Последовательность исполнения

7.1. Операционный алгоритм

1. Создать run_id, каталог запуска, Budget и версию конфигурации. Проверить доступность записи до начала сети.
2. Вызвать normalize_input(). При ошибке сохранить failed и INPUT_*; закончить без HTTP.
3. Сформировать allowed_hosts и исходный кандидат. Проверить его назначение и возможность фактического безопасного соединения.
4. Выполнить fetch_with_policy(): GET, ограниченные повторы, ручные редиректы, размер, дедлайн и журнал попыток.
5. Если ответ пригоден, выбрать его. Если получен подходящий транспортный сбой HTTPS, проверить условия fallback и ровно один HTTP-кандидат. Если смена схемы не поможет, закончить без лишней попытки.
6. Классифицировать HTTP-ответ и содержимое. Сформировать status, can_enter_step1, reason_codes и next_action по таблице решений.
7. Сохранить тела, манифест, site.json и журнал запуска. Провал записи не скрывать.
8. Если can_enter_step1=true, передать шагу 1 выбранный FetchResult, evidence_id, общий Budget и область. Уже полученный входной HTML повторно не загружать без необходимости.

7.2. Псевдокод

```
run = init_run(config)
try:
    entry = normalize_input(input_url)
    scope = build_scope(entry, config.allowed_hosts)
    first = fetch_with_policy(entry, scope, budget)
    attempts = [first]
    if should_try_http(first, entry, config, budget):
        attempts += [fetch_http_sibling(entry, scope, budget)]
    selected = select_usable_response(attempts)
    result = classify_step0(entry, selected, attempts)
except KnownInputOrNetworkError as error:
    result = failure_from(error, preserved_attempts)
persist_evidence_then_result(result)
return result
```

7.3. Интерфейс будущего Python-модуля

Функция validate_site(input_url, config, client, budget) возвращает типизированный SiteValidationResult. Клиент и бюджет инъецируются; это позволяет тестировать ветви без интернета. Сетевые исключения переводятся в данные на границе модуля, а программные ошибки не маскируются фразой «сайт недоступен».

CLI выводит status, конечный URL, причину и папку результатов. При needs_input сообщает точный целевой хост и параметр повторного запуска. Автоматически запускать следующий платный этап CLI не должен. Сериализованный результат пригоден для будущего интерфейса без разбора текста консоли.

8. Приёмочные сценарии

8.1. Обязательная проверка перед приёмкой кода

Сценарий	Ожидаемый результат
Домен без схемы, исправный HTTPS	Один выбранный HTTPS HTML; fallback=false
Явный HTTP → HTTPS	Конечный HTTPS и сохранённый redirect
TLS failure, HTTP с HTML	partial; ошибка TLS сохранена; переход в шаг 1 разрешён
HTTP → тот же неисправный HTTPS	Нет цикла; fallback результата не дал
Явный HTTPS вернул 403	HTTP fallback не отправлен
Вход /ru/?city=msk	Путь/query сохранены; fragment отсутствует
IDN и регистр пути	Корректный сетевой хост; исходный путь не изменён
Наблюдаемый допустимый www-редирект	Алиас сохранён с основанием
Чужой хост в Location	needs_input; целевой GET не выполнен
Private IPv4/IPv6 или DNS сменил назначение	Запрос во внутреннюю сеть не выполнен
200 с CAPTCHA / 204 / бинарный файл	Нет ложного положительного результата
Чрезмерный размер / бесконечный поток	Ограниченная остановка и структурированная причина
Ошибка записи raw/	Нет успешного site.json с несуществующей ссылкой

Все ветви проверяются контролируемым транспортом и фикстурами. Для TLS нужны тесты именно классификации ошибки сертификата; нельзя заменять их проверкой HTTP-кода 500. Счётчики сравниваются с фактическим числом запросов транспорта. Тест на внешний редирект проверяет отсутствие запроса назначения, а не только текст ошибки.

8.2. Критерий готовности

Реализация готова, когда перечисленные сценарии проходят, доказательства разрешаются, общий бюджет не превышается, входной URL не теряет смысл и шаг 1 получает сохранённый ответ. Живой smoke test допустим отдельно на разрешённом сайте; он не заменяет воспроизводимые ветвевые проверки и не объявляется выполненным без реального запуска.

9. Решения, связь с шагом 1 и источники

9.1. Что закреплено этой редакцией

Исправления, обсуждённые с пользователем, собраны в рабочую редакцию для дальнейшей разработки: HTTPS-first, явный HTTP fallback, сохранение пути, ручные контролируемые редиректы, проверка содержимого и доказательств. Начальные лимиты являются конфигурацией проекта. Изменение лимита не должно незаметно менять смысл ранее сохранённого результата.

В этом документе дополнительно уточнены детали раннего черновика: can_enter_step1 отделено от разрешения обхода; DNS NXDOMAIN не запускает бессмысленную смену схемы; для шага 0 введены подлимит 15 HTTP-попыток и 60 секунд внутри общего бюджета. Эти уточнения относятся к редакции 1.0 и фиксируются при её реализации.

9.2. Передача следующему этапу

Шаг 1 получает site.json, выбранный исходный HTML, состояние бюджета, доказательства и разрешённые хосты. Он читает robots.txt для нужного origin, определяет разрешение своего обходчика, выбирает остальные страницы и формирует crawl.json/checks.json. Ошибки HTTPS из шага 0 повторно не вычисляются без причины и остаются видны в техническом отчёте.

Пример: сайт читается по HTTP, а HTTPS имеет неверный сертификат. Шаг 0 завершается partial и передаёт HTML. Шаг 1 может получить полезный текст и успешно закончить чтение; проверка HTTPS при этом всё равно остаётся fail. Готовность материала для профиля и исправность транспорта - независимые результаты.

9.3. Источники

Исходная основа: пользовательский «Алгоритм GEO-аудита сайта», 9 страниц, 31.10.2026. Локальная неизменённая копия находится в docs/source/. Проектные уточнения: PROJECT_CONTEXT.md, ALGORITHM_REVIEW.md и STEP_0_1_SPEC.md.

S1. HTTPX: SSL - проверка сертификатов и конфигурация доверия.

S2. HTTPX: QuickStart - GET, редиректы и история переходов.

S3. HTTPX: Timeouts - сетевые таймауты, отличающиеся от полного дедлайна запуска.

S4. HTTPX: Transports - изоляция сетевого слоя и воспроизводимые mock-тесты.

Документация проверена в ходе подготовки проекта 31.10.2026. Названия полей, коды ошибок, границы хостов, лимиты и условия перехода являются решениями нашей методики, а не прямыми требованиями HTTPX. В документе нет результатов живого аудита и нет утверждения, что Python-реализация уже создана.

9.4. Управление изменениями

Каждое изменение правил нормализации, fallback, области или пригодности ответа повышает algorithm_version. Сохранённый запуск содержит старую версию и конфигурацию. Исходные доказательства не перезаписываются; повторный анализ создаёт новую производную запись и сохраняет связь с первым наблюдением.